Week 2 Module: Programming practice

- Language Used: Java

1. Fraud Transaction Detection (Linear Search)

Problem Statement

A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

Write a program using Linear Search.

Input Format

- First line contains integer N
- Second line contains N transaction IDs
- Third line contains suspicious transaction ID

Logic

1. Start the program.
2. Read the number of transaction IDs from the user.
3. Read All Transaction IDs into an array.
4. Read the suspicious transaction ID from the user.
5. Use a loop to iterate the array and find the suspicious ID.
6. If the ID is found:
    - Print Fraud Transaction Found.

   Else:

    - Print transaction not found.
7. End the program.

Source Code

```
 import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class LinearSearch{
   public static void main(String[] args) {
      try (Scanner scan = new Scanner(System.in)) {
         System.out.print("Enter the number of transaction IDs: ");
         int n = scan.nextInt();
         List<Integer> lst = new ArrayList<>();
         System.out.println("Enter transaction IDs: ");
         for (int i = 0; i < n; i++) {
```

```
            int id = scan.nextInt();
            if (lst.contains(id)) {
               System.out.println("Transaction " + "'" + id + "'" + " already exists");
               System.out.println("---Enter another transaction ID----");
               i--;
            }
            else {
               lst.add(id);
            }
         }
         System.out.println("Enter Suspicious ID: ");
         int sus = scan.nextInt();
         int res = linearSearch(lst, sus);
         if (res == -1) {
            System.out.print("----Transaction not found----");
         }
         else {
            System.out.print("----Fraud transaction found----");
         }
      }
      catch (Exception e) {
         System.out.println("Invalid Input");

      }
   }
   public static int linearSearch(List<Integer> lst, int x) {
      for (int i = 0; i < lst.size(); i++) {
         if (lst.get(i) == x) {
            return i;
         }
      }
      return -1;
   }
}
```

Output

```
Enter the number of transaction IDs: 5
Enter transaction IDs:
1201 1305 1450 1408 1502
Enter Suspicious ID:
1450
----Fraud transaction found----
Process finished with exit code 0
```

2. Server Log Search (Binary Search)

Problem Statement

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists.

Write a program using Binary Search.

Input Format

- First line contains N
- Second line contains sorted log IDs
- Third line contains target log ID

Logic

1. Start the program.
2. Read the number of server log IDs from the user.
3. Read the sorted log IDs into an array.
4. Read the target log ID from the user.
5. Use a while loop to iterate through the array.
6. Find the middle element in the array:
    - If the target ID is greater than middle element:
        - Then shorten array starting from the middle+1 element till the array length
    - If the target ID is less than the middle element:
        - Then shorten the array length to the middle element-1.

    Repeat this until the middle element is the targeted ID.
7. If the target ID is found then print “Log is Found” else print “Log not found”.
8. End the program.

Source Code

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class BinarySearch{
   public static void main(String args[]){
      try(Scanner scan = new Scanner(System.in)){

         System.out.print("Enter the number of log IDs: ");
         int n = scan.nextInt();
         List<Integer> lst = new ArrayList<>();
         System.out.println("Enter log IDs: ");
         for (int i = 0; i < n; i++) {
            int id = scan.nextInt();
            if (lst.contains(id)) {
               System.out.println("Log " + "'" + id + "'" + " already exists");
               System.out.println("---Enter another log ID----");
               i--;
            }
            else {
               lst.add(id);
            }
         }
         System.out.println("Enter Suspicious ID: ");
         int sus = scan.nextInt();
         int res = binarySearch(lst, sus);
         if (res == -1) {
            System.out.print("----Transaction not found----");
         }
         else {
            System.out.print("----Fraud transaction found----");
         }
      }
      catch (Exception e){
         System.out.println(e.getMessage());
      }
   }
   public static int binarySearch(List<Integer> lst, int sus){
      int low = 0;
      int high = lst.size()-1;
      while (low <= high){
         int mid = (low+high)/2;
         if (lst.get(mid).equals(sus)){
            return mid;
         }
         else if (lst.get(mid).compareTo(sus) <= 0){
```

```
            low = mid+1;
        }
        else if (lst.get(mid).compareTo(sus) >= 0){
            high = mid-1;
        }
    }
    return -1;
  }
}
```

Output

```
Enter the number of log IDs: 7
Enter log IDs:
1001 1005 1010 1015 1020 1030 1040
Enter Suspicious ID:
1015
----Fraud transaction found----
Process finished with exit code 0
```

3. Sort Employee Salaries (Bubble Sort)

Problem Statement

A company wants to sort employee salaries in ascending order for payroll analysis.

Write a program using Bubble Sort.

Input Format

- First line contains N
- Second line contains salaries

Output Format

Print sorted salaries.

Logic

1. Start the program.
2. Read the number of employees from the user.
3. Read all the salaries into an array.
4. Use an outer loop to iterate through the salaries.
5. Use an inner loop to swap salaries
    - If a salary is greater than the next salary in the array then swap those salaries else continue the loop
6. Repeat the process until array is sorted.
7. End the program.

Source Code

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;


class BubbleSort {
   void main(){
      try(Scanner scan = new Scanner(System.in)){
         List<Integer> list = new ArrayList<>();
         System.out.print("Enter number of employees: ");
         int n = scan.nextInt();
         System.out.println("----Enter salaries----");
         for(int i = 0; i < n; i++){
            int salary = scan.nextInt();
            if(salary < 0){
               System.out.println("Salary cannot be negative");
               System.out.println("Re-enter salary");
               i--;
            }
            else {
               list.add(salary);
            }
         }
         bubbleSort(list, list.size()-1);
         display(list);
      }
      catch(Exception e){
         System.out.println("Invalid Input");
      }
   }
```

```
    public void bubbleSort(List<Integer> list, int size){
        for(int i = 0; i < size; i++){
            boolean swapped = false;
            for(int j = 0; j < size-i; j++){
                if(list.get(j) > list.get(j+1)){
                    int temp = list.get(j);
                    list.set(j, list.get(j+1));
                    list.set(j+1, temp);
                    swapped = true;
                }
            }
            if(!swapped){
                break;
            }
        }
    }

    public void display(List<Integer> list){
        for (Integer salary : list) {
            System.out.print(salary + " ");
        }
    }
}
```

Output

```
Enter number of employees: 5
----Enter salaries----
45000 32000 55000 28000 60000
28000 32000 45000 55000 60000
Process finished with exit code 0
```

4. Sort Website Response Time (Insertion Sort)

Problem Statement

A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position.

Write a program using **Insertion Sort**.

Input Format

- First line contains N

- Second line contains response times

Output Format

Print sorted response times.

Logic

1. Start the program.
2. Read the number of response time from the user.
3. Read all the response time into an array.
4. Use an outer for loop to iterate through the array.
5. Use a while loop to divide the array into sorted and unsorted part.
6. Continuously check in the while for a sorted element with unsorted element
    - If the unsorted element is smaller than the sorted element then place the unsorted element into the correct sorted order.
    - Else continue.
    - Do this until array is sorted.
7. End the program.

Source Code

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class InsertionSort {
   void main(){
      try(Scanner scan = new Scanner(System.in)){
         List<Integer> list = new ArrayList<>();
         System.out.print("Enter number of Response time: ");
         int n = scan.nextInt();
         System.out.println("----Enter Response times----");
         for(int i = 0; i < n; i++){
            int rTime = scan.nextInt();
            if(rTime < 0){
               System.out.println("Response time cannot be negative");
               System.out.println("Re-enter Response Time");
               i--;
            }
            else {
               list.add(rTime);
```

```
            }
        }
        insertionSort(list, list.size());
        display(list);
    }
    catch(Exception e){
        System.out.println("Invalid Input");
    }
}

public void insertionSort(List<Integer> list, int size){
    for(int i = 1; i < size; i++){
        int temp =  list.get(i);
        int j = i-1;
        while(j>=0 && list.get(j)>temp){
            list.set(j+1, list.get(j));
            j--;
        }
        list.set(j+1, temp);
    }
}

public void display(List<Integer> list){
    for (Integer rTime : list) {
        System.out.print(rTime + " ");
    }
}
}
}
```

Output

```
Enter number of Response time: 5
----Enter Response times----
250 120 320 180 100
100 120 180 250 320
Process finished with exit code 0
```

5. Prioritize Bug Reports (Selection Sort)

Problem Statement

A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

Write a program using **Selection Sort**.

Input Format

- First line contains N
- Second line contains priority scores

Output Format

Print sorted priority scores.

Logic

1. Start the program.
2. Read the number of priority scores from the user.
3. Read all the scores into an array.
4. Use an outer loop to iterate through the array.
5. Use an inner loop to find min element and swap it at the start, sorting the array from the start
6. Repeat the process until array is sorted.
7. End the program.

Source Code

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class SelectionSort {
   void main(){
      try(Scanner scan = new Scanner(System.in)){
         List<Integer> list = new ArrayList<>();
         System.out.print("Enter number of Priorities: ");
         int n = scan.nextInt();
         System.out.println("----Enter Priorities----");
         for(int i = 0; i < n; i++){
            int prScore = scan.nextInt();
            if(prScore < 0){
               System.out.println("Priority cannot be negative");
               System.out.println("Re-enter Priority");
               i--;
            }
            else {
               list.add(prScore);
            }
         }
```

```
            selectionSort(list, list.size());
            display(list);
        }
        catch(Exception e){
            System.out.println("Invalid Input");
        }
    }

    public void selectionSort(List<Integer> list, int size){
        for(int i = 0; i < size; i++){
            int min =  list.get(i);
            for (int j = i+1; j < size; j++){
                if(list.get(j) < min){
                    min = list.get(j);
                    int temp = list.get(i);
                    list.set(i, min);
                    list.set(j, temp);
                }
            }
        }
    }

    public void display(List<Integer> list){
        for (Integer prScore : list) {
            System.out.print(prScore + " ");
        }
    }
}
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}
```

Output

```
Enter number of Priorities: 5
----Enter Priorities----
9 2 8 1 5
1 2 5 8 9
Process finished with exit code 0
```